

COL100/COL1000 Assignment 2

Submission Instructions

Write your solution in the provided `assignment.py` starter file. The function signatures are already defined. Do **not** add new functions or change the existing function names, parameters, or return types. Simply replace each `pass` statement with your implementation.

To test your solution against the visible test cases, place both `assignment.py` and `test_visible.py` in the **same folder** and run:

```
python test_visible.py
```

or, if your system requires it:

```
python3 test_visible.py
```

Note that passing all visible test cases does **not** guarantee full marks. Your code will also be evaluated against hidden test cases.

You can submit **only** the `assignment.py` file, exactly as named, without renaming or restructuring it.

Question 1: Power of a Number

Write a recursive Python function:

```
def power(x: int, y: int) -> int:
```

The function calculates the value of a base x raised to the power of y (where y is a non-negative integer).

Validation Requirement:

1. At the very beginning of your function (before any logic), print the current function call exactly like this:

```
print(f"power({x}, {y})")
```

2. To output the final result of the operation, ensure that your code prints the final answer exactly once before returning it (you may use a wrapper function if needed to avoid printing on every recursive step). Use exactly this template:

```
print(f"Final Result: {result}")
```

Constraint: Do not use built-in math power functions.

Test Cases

Test Case 1

Function call:

```
power(2, 3)
```

Expected Standard Output (Printed to console):

```
power(2, 3)
power(2, 2)
power(2, 1)
power(2, 0)
Final Result: 8
```

Question 2: The Ping-Pong Parity (Mutual Recursion)

Write two distinct recursive Python functions:

```
def isEven(n: int) -> bool:
def isOdd(n: int) -> bool:
```

- `isEven` should return `True` if n is 0, and otherwise return `isOdd(n-1)`.
- `isOdd` should return `False` if n is 0, and otherwise return `isEven(n-1)`.

Add logic to handle negative numbers gracefully (e.g., using absolute values) so the call stack doesn't overflow.

Validation Requirement:

1. At the beginning of `isEven(n)`, print exactly: `print(f"isEven({n})")`
2. At the beginning of `isOdd(n)`, print exactly: `print(f"isOdd({n})")`
3. To output the final boolean result of the operation, print it exactly like this at the end of your execution:

```
print(f"Final Result: {result}")
```

Test Cases

Test Case 1

Function call:

```
isEven(4)
```

Expected Standard Output:

```
isEven(4)
isOdd(3)
isEven(2)
isOdd(1)
isEven(0)
Final Result: True
```

Question 3: Count Paths in a Grid with Obstacles

Write a recursive Python function:

```
def count_paths(grid: list) -> int:
```

Imagine a robot sitting in the top-left corner of an $M \times N$ grid. The robot can only move right or down at any point. Some cells in the grid contain obstacles (represented by 1), while empty spaces are represented by 0. Write a recursive function to count how many unique paths the robot can take to reach the bottom-right corner.

The Goal: Navigate 2D recursive space with boundary conditions and obstacle checks.

Validation Requirement: To output the final result of the operation, print it exactly like this at the end of your execution (using a wrapper function is recommended):

```
print(f"Final Result: {result}")
```

Test Cases

Test Case 1

Function call:

```
grid = [
    [0, 0, 0],
    [0, 1, 0],
    [0, 0, 0]
]
count_paths(grid)
```

Expected Standard Output:

```
Final Result: 2
```

Question 4: Recursive Palindrome Checker (Two-Pointer Recursion)

Write a recursive Python function:

```
def is_palindrome(s: str) -> bool:
```

This problem introduces the concept of shrinking the problem space from multiple directions at once.

The function should check if a given string is a palindrome (it reads the same forwards and backwards). For an added challenge, your function should ignore spaces, punctuation, and capitalization.

Validation Requirement:

1. At each recursive step (assuming the string has been cleaned of spaces/punctuation and converted to lowercase), if the current string has at least two characters to compare, print the outermost characters being compared exactly like this:

```
print(f"Comparing: '{first_char}' and '{last_char}'")
```

2. To output the final boolean result of the operation, print it exactly like this at the end of your execution:

```
print(f"Final Result: {result}")
```

Test Cases

Test Case 1

Function call:

```
is_palindrome("A man a plan a canal Panama")
```

Expected Standard Output:

```
Comparing: 'a' and 'a'
Comparing: 'm' and 'm'
Comparing: 'a' and 'a'
Comparing: 'n' and 'n'
Comparing: 'a' and 'a'
Comparing: 'p' and 'p'
Comparing: 'l' and 'l'
Comparing: 'a' and 'a'
Comparing: 'n' and 'n'
Comparing: 'a' and 'a'
Final Result: True
```

Question 5: Generate All Subsets (Power Set)

Write a recursive Python function:

```
def generate_subsets(nums: list) -> list:
```

Given an array of unique integers `nums`, the function should return all possible subsets.

Validation Requirement: For this question, **skip printing intermediate states or recursive steps**. You must only print the final list of subsets generated by the algorithm exactly once before returning it.

Use exactly this format:

```
print(f"Final Result: {result}")
```

Test Cases

Test Case 1

Function call:

```
generate_subsets([1, 2])
```

Expected Standard Output (Note: The exact order of the inner subsets may vary depending on whether your recursion branches to "include" or "exclude" first, but the list must contain exactly 4 subsets):

```
Final Result: [[1, 2], [1], [2], []]
```